

05 — Build it from scratch

A milestone-by-milestone roadmap for recreating ProvenanceRank yourself. Each milestone produces something you can *run and verify* before moving on. You don't need the heavy libraries to start — build the pure-Python core first, then layer the smart bits on top.

Mindset: build the simplest thing that produces a valid `submission.csv`, verify it, then improve accuracy, then wrap it in a service. Never let a fancy dependency become a hard requirement — always have a fallback.

Prerequisites

- **Python 3.11+**. Create a virtual environment: `python -m venv .venv && source .venv/bin/activate`.
 - Start with just `numpy`, `pandas`, `joblib`. Add the rest later.
 - (For the UI, much later) **Node 18+**.
 - Get the dataset (`candidates.jsonl`) and a `job_description.txt`.
-

Milestone 0 — Project skeleton

Create the folders from [01_overview.md](#)'s repo map and a `core/config.py` that reads settings from environment variables with sensible defaults (paths, weights, model names). Add `core/logging.py` for JSON logs.

Why first: one place for configuration and logging means every later module stays clean. **Verify:** `python -c "from core.config import get_settings; print(get_settings().top_k)"` prints 100.

Milestone 1 — Load data and build features

1. `pipeline/loader.py`: a generator that yields one JSON profile per line, plus a `corpus_text()` that flattens a profile into one string.
2. `core/constants.py`: define `NUMERIC_FEATURE_COLUMNS` — the list of ~40 feature names. This is your **contract**: every model agrees on these columns.
3. `pipeline/feature_engineering.py`: `build_feature_row(candidate) -> dict` that computes those 40 numbers (experience, tenure, skill matches, assessment scores, behavioural composite, location fit...), and `frame_from_rows()` to stack them into a pandas DataFrame.

Verify: stream 100 candidates, build the frame, print `df.shape` → (100, ~40+). Check there are no NaNs in the numeric columns.

Milestone 2 — Honeypots and gates

1. `pipeline/honeypot_detector.py`: a handful of **strict** rules that flag impossible profiles (e.g. summed role durations exceed a plausible career length). Optimise for *precision* — never flag a real person.
2. `pipeline/gate_filter.py`: `apply_gates(df, spec)` that marks candidates failing hard requirements (consulting-only, keyword-stuffer, inactive). Keep them in the frame but flagged, and write the reasons to `excluded_candidates.csv`.

Verify: on the real data you should see roughly ~65 honeypots flagged and tens of thousands gated — and zero honeypots in your final top 100.

Milestone 3 — Retrieval (find the relevant candidates)

1. `pipeline/bm25_indexer.py`: build a BM25 keyword index over the profile texts. Start with the `rank-bm25` library; later add a streaming inverted-index fallback if memory is tight.
2. `pipeline/embedder.py`: encode profiles into vectors. Start with the **hashing embedder** (pure NumPy — no torch needed): hash each token to a bucket and sign, sum, L2-normalise. Add sentence-transformers later as the high-quality path.
3. `pipeline/retrieval.py`: `cosine_scores()` for the dense side and `fuse_rrf()` to combine BM25 ranks and embedding ranks via Reciprocal Rank Fusion.

Verify: for the JD, print the top 10 by `retrieval_score`. They should look topically relevant (titles/skills matching the JD).

Milestone 4 — The two entry points and a first submission

1. `pipeline/scorer.py`: `merge_final_scores()` — start simple: $0.40 \cdot \text{ml_fit} + 0.35 \cdot \text{retrieval} + 0.25 \cdot \text{behavioural}$, then multiply by location and availability factors, and force honeypots/gated to 0.
2. `ml/trainer.py` + `ml/predictor.py`: with no labels, build **proxy labels** from features you trust and fit a gradient-boosted regressor (or just return the proxy as a formula). `predict_fit()` loads the model, or falls back to the formula.
3. `output/reasoning_generator.py`: a template that writes one sentence per pick.
4. `output/submission_writer.py`: write exactly 100 rows and run the validator.
5. `precompute.py`: stream once → features → gates → embeddings → BM25 → retrieval → train → save artifacts.
6. `rank.py`: load artifacts → predict → merge → drop honeypots → top 100 → reasons → write + validate.

Verify: `python precompute.py ...` then `python rank.py ...` then `python validate_submission.py submission.csv` → **“Submission is valid.”** You now have a working ranker. Everything after this is making it *better* and *bigger*.

Milestone 5 — Measure, then improve accuracy

1. `ml/evaluate.py`: implement NDCG@k, MAP, P@k, and the weighted composite. Without this you’re guessing.
2. `ml/pseudo_labels.py`: build **graded** labels (tiers 0–5) by bucketing a rich composite by percentile. (Optional: have Gemini grade a sample.)
3. Upgrade `ml/trainer.py` with `train_ranker()` — an XGBRanker with `objective="rank:ndcg"` (LambdaMART) trained on those graded labels.
4. `ml/reranker.py`: a cross-encoder that re-scores the top ~300 (with a lexical fallback). Cache `rerank_score`.
5. `pipeline/tournament.py`: a Bradley-Terry pairwise tournament over the top ~60. Cache `tournament_score`.
6. Fold both into `scorer.py` as a convex head-blend, and wire all of it into `precompute.py`.

Verify: use the eval harness to A/B baseline vs. the accuracy layer on the graded labels — the composite should go up and the top-20 should reorder.

Milestone 6 — Make the offline phase fast

1. `core/device.py`: auto-detect MPS/CUDA; optional ONNX/int8 toggle.

2. `core/artifact_cache.py`: hash each stage's inputs; reuse the artifact on a re-run when the key matches. Wire it into `precompute.py` so changing only the JD reuses the embeddings.
3. In `trainer.py`: train on all positives + sampled negatives, histogram trees, early stopping.

Verify: a second `precompute --resume` with unchanged inputs finishes in ~1s; changing only the JD reuses the embeddings (look for `cache.hit` in the logs).

Milestone 7 — Wrap it in a production API

1. `db/` — SQLAlchemy 2.0 async models + a repository layer + Alembic migrations. Use SQLite for dev (it's a fallback for Postgres).
2. `core/security.py` + `api/routers/auth.py` — bcrypt passwords, JWTs, API keys.
3. `services/ranker.py` — load the engine once, keep it warm.
4. `services/queue.py` + `services/jobs.py` — a job queue + worker; the request returns a `job_id`, progress streams over SSE.
5. `services/cache.py` + `services/ratelimit.py` — Redis with in-memory fallback.
6. `api/` — the FastAPI app, routers, schemas, middleware, error handlers.
7. `core/logging.py`, `api/metrics.py`, `core/tracing.py`, `core/circuit_breaker.py` — observability + resilience.

Verify: `uvicorn api.main:app` → open /docs, log in as the bootstrap admin, submit a ranking job, watch it stream to completion.

Milestone 8 — The live signal ingestion layer

1. `intelligence/confidence_scorer.py` — the source-weighted, recency-decayed trust formula. (Build this first; it's the heart of the layer.)
2. `intelligence/skill_extractor.py` + `summariser.py` — canonicalise skills, summarise artifacts (both with heuristic fallbacks).
3. `ingestion/github_client.py` + `sync.py` + `tasks.py` — pull GitHub activity (async, rate-limited, circuit-breaker-guarded), turn it into evidence.
4. `graph/schema.py` + `queries.py` + `natural_language_query.py` — a SHA-256-anchored proof-of-work graph (Neo4j or in-memory) and an English-to-query endpoint.
5. Compose it back: `feature_engineering.apply_live_signals()` adds an additive `evidence_confidence_bonus` (default 0) that `scorer.py` includes — so the graded submission is unchanged when the layer is off.

Verify: without a GitHub token everything no-ops cleanly; with one, ingest a developer and query “who has shipped production LLM systems?” in plain English.

Milestone 9 — Frontend, Docker, CI

1. `frontend/` — a React + TypeScript + Vite app (Tailwind, React Query, router): Login, Dashboard, Ranking Results, NL Search, Settings; an SSE hook for live progress.
2. `Dockerfile` + `docker-compose.yml` — multi-stage images and the full stack (Postgres, Redis, Neo4j, API replicas, nginx gateway, workers, Prometheus, Grafana, Jaeger, the UI).
3. `.github/workflows/ci.yml` — install, lint with Ruff, run pytest.

Verify: `docker compose up --build` brings the whole system up; the UI talks to the API through the nginx gateway.

The order, in one line

features → honeypots/gates → retrieval → first valid submission → measure → accuracy layer → speed → API → live layer → UI/Docker/CI.

Build each milestone with its fallback path first; add the heavy library as an upgrade, never a requirement. That single rule is what lets the finished project run anywhere from a bare sandbox to a full GPU cluster.

See [06_glossary.md](#) for any unfamiliar term.